Technical Note

Power Electronics and Control Systems

PMSM wind generator with predictive current control and extended Kalman filter

the model `mpc_psm`: two-level inverter, finite-set MPC with delay compensation, sensorless EKF

How the model `foundations_of_power_electronics/mpc_psm` of `pwr-control/modelization-and-control` works after the control fixes of November 2026: the machine seen from the bridge, the predictive control and its current loop, the extended Kalman filter and its speed, the torque reference, the parameters and the results of the design check.

Authors:	**Revision:**	**Date:**
Davide Bagnara	01	November 2026

Abstract: The model `mpc_psm` simulates one of the six three-phase systems of a 1.6 MW, 17.8 rpm, 104-pole permanent magnet wind generator (about 267 kW per system) on a two-level inverter with a 230 µH output inductor and a DC link across a 1200 V lithium-ion battery. A finite-set model predictive control (FCS-MPC) tests the eight states of the bridge every 66.7 µs and writes the six gate commands; an extended Kalman filter (EKF) gives the angle and the flux of the rotor without a position sensor, and a double integrator observer on the EKF angle gives the speed. This note describes the model after the control fixes of November 2026: the machine seen from the bridge and its active flux, which makes one inductance (L_q plus the inductor) right for the control; the voltage of the states, the delay compensation and the cost function with its switching term; the current loop seen as a deadbeat loop; the six-state EKF, its state prediction, noise and input timing, and why its speed state is 0.3 % low with the forward Euler step; the torque reference. A switched simulation of the whole chain checks the design: sensorless angle error $-0.1°$ (std $0.35°$), speed 0.9992, current error 2.8 A rms, 2.8 kHz per device, sensorless from -25 % to $+20$ % of the machine inductances. Parameters, files, links to the model and open points close the note.



Revision history

Rev.	Date	Description	Authors
00	before Nov. 2026	The earlier document of this folder: PMSM and induction machine models, EKF, space vector and predictive modulators of an older control (no date; its title was the one of `im_mpc_ekf`).	Davide Bagnara
01	November 2026	Complete rewrite on the model `mpc_psm` after the control fixes of November 2026 (modelization-and-control PR #33); the induction machine part is left out (see `im_mpc_ekf`); companion scripts of the design check.	Davide Bagnara

MATLAB model: modelization-and-control.git (foundations_of_power_electronics/mpc_psm)

Framework: MATLAB/Simulink, Simscape, GNU Octave, Python



Contents

Nomenclature 3

1. Introduction 4

2. The model 5
2.1. Power stage . . . 5
2.2. Control chain . . . 6
2.3. Timing of one control step . . . 7

3. The machine seen from the bridge 7
3.1. Per-unit bases . . . 7
3.2. Model in the rotor frame . . . 8
3.3. Active flux: one inductance for the control . . . 8

4. Finite-set predictive current control 9
4.1. Voltage of a state . . . 9
4.2. Prediction and delay compensation . . . 10
4.3. Cost function and switching term . . . 10
4.4. The current loop as a deadbeat loop . . . 11
4.5. Gates and the voltage for the observers . . . 12

5. Torque and current reference 12

6. Extended Kalman filter 13
6.1. State, model and output . . . 13
6.2. Jacobian . . . 14
6.3. Prediction of the state and of the covariance . . . 14
6.4. Correction and noise . . . 15
6.5. Voltage and current given to the filter . . . 15
6.6. Angle, flux and speed: the double integrator observer . . . 15
6.7. Sensitivity to the machine inductances . . . 16

7. Design check 16
7.1. How the check works . . . 16
7.2. Results . . . 17

8. What changed in November 2026 19

9. Parameters 20

10. Files and how to run the model 21

11. Open points 22

A. The function `mpc` of the model 25

List of Figures

1. The model `mpc_psm`: power stage (top) and control (bottom). The control runs every $t_s = 66.7\,\mu s$ in pu of the machine; $\underline{u}_{k-2}$ is the voltage of the vector applied two steps before. . . . 5



2. Timing of one control step: the samples taken at t_{k-1} are used at step k; the vector chosen at step k acts over the next step. 7
3. The eight states of the two-level bridge in the $\alpha\beta$ plane (pu of u_{bez} at 1249 V), with their number in the table of the code; the two zero states are at the origin. Blue: largest sinusoidal voltage (linear limit $v_{\text{dc}}/\sqrt{3}$, 1.97 pu); orange: voltage of the machine at rated speed (1.30 pu). Dashed: the regions of the voltages nearest to each state, which the cost function picks (Section 4.3). 10
4. Torque curve of the model per system (`param.load.rpm_base`, `torque_base`: the curve of the whole machine divided by six). 13
5. Start-up and rated operation in the switched simulation: q and d current, error of the control angle and speed of the control; model before the fix (A), fixed model sensorless (N1) and with the encoder (N2). 18
6. Current i_α and its reference over 100 ms at rated speed: model before the fix with and without the bridge passives (A, A0), fixed model (N1) and fixed model with $k_{\text{sw}} = 10^{-4}$ (S2). 19

List of Tables

1. Where to find the model. 4
2. Power stage of `mpc_psm.slx` (one system of the generator). 6
3. Bases and per-unit values of one system (`param.psm.machine`, `param.inv.hw`). . 8
4. Current loop seen as a deadbeat loop on one axis (no vector limit). 12
5. Non-zero entries (row, column) of the coefficient matrices of the Jacobian; states in the order $i_\alpha, i_\beta, \psi_\alpha, \psi_\beta, \omega, \tau_{\text{L}}$. 14
6. Isolated test of the EKF at rated speed (`py/ekf_speed_bias.py`): exact plant, deadbeat voltage, no switching. 16
7. Results of the switched simulation (17.8 rpm unless stated, averages over 1.0–1.25 s). Angle: error of the control angle (std); speed: speed of the control over the true speed; torque: against the torque curve at the true speed; switching: leg changes per second / 2. 17
8. Control fixes of November 2026. 20
9. Control parameters of `mpc_psm` (`init_model.m`, `private/psm_ctrl_param.m`). . . 21
10. Files of the model and of this note. 22



Nomenclature

Symbol	Meaning
PMSM	permanent magnet synchronous machine
FCS-MPC	finite-set model predictive control: at every step it tests the states of the bridge and applies the best one
EKF	extended Kalman filter
DIO	double integrator observer: angle and speed from an angle signal
pu	per unit of the machine (one system): voltages in u_{bez}, currents in i_{bez}, time in $1/\omega_{\mathrm{bez}}$
u_{bez}, i_{bez}	base voltage (peak phase voltage at rated speed, $\psi_{\mathrm{m}}\,\omega_{\mathrm{bez}}$) and base current (peak rated phase current) of one system
ω_{bez}, τ_{bez}	electrical base pulsation (rated speed) and base torque of one system
L_{bez}, X_{bez}	base inductance and impedance, $X_{\mathrm{bez}} = u_{\mathrm{bez}}/i_{\mathrm{bez}} = \omega_{\mathrm{bez}} L_{\mathrm{bez}}$
$u_{\mathrm{dc,nom}}$	1070 V, base of the measured DC voltage v_{dc} (data set `inv690V_250kW`)
k_u	$u_{\mathrm{dc,nom}}/u_{\mathrm{bez}} = 2.925$, from pu of $u_{\mathrm{dc,nom}}$ to pu of u_{bez} (`u_scale`)
$\alpha\beta$, dq	stationary frame (amplitude-invariant Clarke transformation) and rotor frame (d on the magnet axis)
$\underline{x}$	space vector $x_\alpha + \mathrm{j}x_\beta$; $\mathrm{j}\underline{x}$ is $\underline{x}$ turned by 90°
$\underline{u}$, $\underline{i}$, $\underline{e}$	voltage of the bridge, current, back-EMF
ψ_{m}, $\underline{\psi}$	magnet flux; flux of the control model (the active flux, Section 3.3)
θ, ω	electrical angle and electrical speed (pu of ω_{bez}) of the rotor
L_d, L_q, R_s	inductances and resistance of the machine (one system)
L_{Fi}, R_{LFi}	output inductor of the inverter and its resistance
L, R	inductance and resistance of the control model: $L = L_q + L_{\mathrm{Fi}}$, $R = R_s + R_{\mathrm{LFi}}$ (`L_norm`, `R_norm`)
t_{s}, k	control step (66.7 µs) and its index; t_k is the time of step k
h	control step in per-unit time, $h = t_{\mathrm{s}}\,\omega_{\mathrm{bez}} = 0.00646$ (`ts_pu`)
$\hat{\gamma}$	coefficient of the prediction, $h/L = 0.00644$ (variable `g` of the code)
g	gain of the current loop: true current change over predicted current change for the same voltage
$s = (s_a, s_b, s_c)$	state of the bridge: $s_x = 1$ when the upper switch of leg x is on, 0 when the lower one is on
J, N_{sw}	cost of a state; number of legs that change state
k_{i}, k_{sw}	weights of the current error and of N_{sw} (`k_kalman`, `k_sw`)
$\boldsymbol{x}$, $\boldsymbol{y}$	state $[i_\alpha\ i_\beta\ \psi_\alpha\ \psi_\beta\ \omega\ \tau_{\mathrm{L}}]^{\mathsf{T}}$ and output $[i_\alpha\ i_\beta]^{\mathsf{T}}$ of the EKF
$A_1 \ldots A_6$, B, C	coefficient matrices of the Jacobian, input matrix and output matrix of the EKF model
F_{pred}, F_{cov}	transition matrix of the state prediction and of the covariance prediction
P, K	covariance of the state error and Kalman gain (the block calls the gain `L(k)`)

Symbol	Meaning
Q_K, R_K	process and measurement noise covariances (`Qkalman`, `Rkalman`)
q_{ekf}, r_{ekf}, j_m	noise weights and inertia of the EKF model (`q_ekf`, `r_ekf`, `jm_ekf`)
k_θ, k_ω	gains of the double integrator observer
T	electromagnetic torque (pu of τ_{bez})

1. Introduction

The model `mpc_psm` [1] belongs to the folder `foundations_of_power_electronics` of `pwr-control/modelization-and-control`. It simulates one of the six three-phase systems of a permanent magnet wind generator (data set WindGen: 1.6 MW, 17.8 rpm, 104 poles, about 267 kW per system) with its two-level inverter, its DC link and a lithium-ion battery (Figure 1). The shaft turns at a fixed speed and the torque reference follows the torque curve of the turbine. The control runs every 66.7 µs (15 kHz) in the per unit of the machine and has no modulator: a finite-set model predictive control (FCS-MPC) [3, 4] tests the eight states of the bridge and writes the six gate commands. The angle and the flux of the rotor come from an extended Kalman filter (EKF) on the measured currents, without a position sensor; the speed comes from a double integrator observer (DIO) on the EKF angle.

This note describes the model as it is in the branch `dev` after the control fixes of November 2026 (pull request #33 of modelization-and-control). It replaces the earlier document of this folder, which described an older version of the control and mixed the PMSM with the induction machine; the induction machine is in the document `repos_doc/im_mpc_ekf` and in the note `research/power_electronics/im_foc_ekf_sensorless` of this repository. The sibling model `mpc_ttype_psm` (same machine, three-level T-type bridge) is described in `research/power_electronics/fcs_mpc_ttype_psm` [9].

Table 1: Where to find the model.

What	Link
model folder (branch `dev`)	https://github.com/pwr-control/modelization-and-control/tree/dev/foundations_of_power_electronics/mpc_psm
README of the model	https://github.com/pwr-control/modelization-and-control/blob/dev/foundations_of_power_electronics/mpc_psm/README.md
init (live script)	`init_model.m` in the model folder: run it from that folder (Section 10)
model	`mpc_psm.slx`; the predictive control is `inner_ctrl/mp control/mpc_pu_d` (MATLAB Function `mpc`), the EKF is `EKF_bemf_observer/extended_kalman_observer`
library	https://github.com/pwr-control/library (blocks of the power stage and of the battery)

Section 2 describes the model and the timing of one control step, Section 3 the machine seen from the bridge in per unit, Section 4 the predictive control, Section 5 the torque and current reference, Section 6 the EKF and the speed, Section 7 the design check, Section 8 what changed

in November 2026, Section 9 the parameters, Section 10 the files and how to run the model, Section 11 the open points. Appendix A lists the function `mpc` of the model.

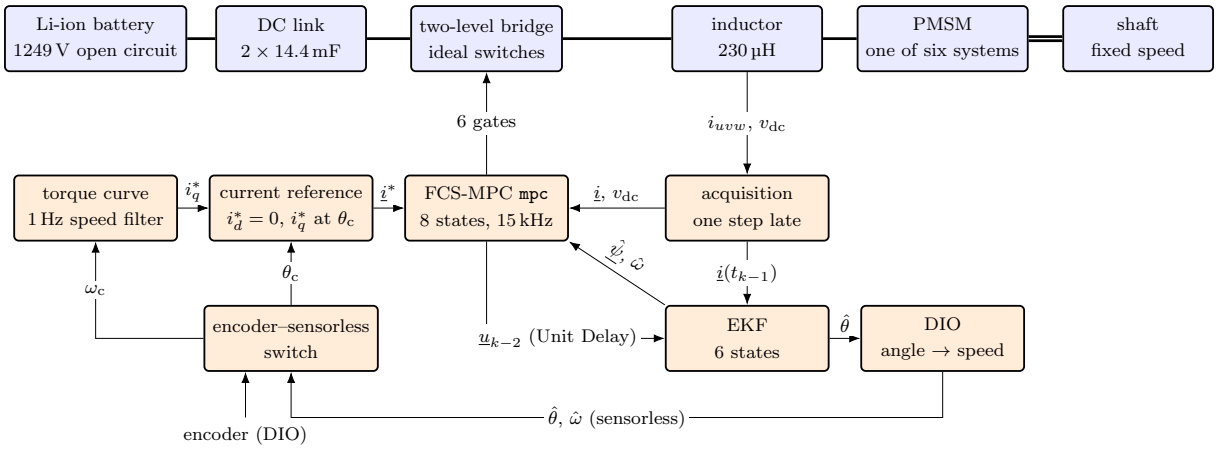


Figure 1: The model `mpc_psm`: power stage (top) and control (bottom). The control runs every $t_s = 66.7\,\mu s$ in pu of the machine; $\underline{u}_{k-2}$ is the voltage of the vector applied two steps before.

2. The model

2.1. Power stage

Table 2 lists the blocks of the power stage. The bridge is the library block `Three-Phase Bridge (ideal)` [2] with its built-in DC capacitor and AC inductors switched off: the model has its own DC link and output inductor. The gate generator `dead_time_generator_cmos_simulink` filters the six gates with the time constant t_c and puts the dead time between the two switches of a leg: the switch that turns on waits until an RC copy (time constant $\sqrt{2} \cdot 3\,\mu s$) of the other gate is below 7.5 V, so the bridge sees $\ln 2 \cdot \sqrt{2} \cdot 3\,\mu s = 2.94\,\mu s$. The solver is `ode23tb`; its maximum step and the step of the Simscape local solver are $t_c = 0.333\,\mu s$, with 5 nonlinear iterations, and the simulation starts from the steady state, so the DC link starts at the battery voltage.

Table 2: Power stage of `mpc_psm.slx` (one system of the generator).

Block	What it is, data
`battery/lithium_ion_battery`	library lithium-ion battery across the whole DC link: 332 cells, 1200 V nominal, 250 kW, 1249 V open circuit at state of charge 0.85; $R_0 = 72\,\mathrm{m\Omega}$, $R_1 = 72\,\mathrm{m\Omega}$, $C_1 = 10\,\mathrm{mF}$ (legacy changes: R_0, R_1 halved, $C_1/50$)
`inv_psm_mod1/dclink`	`dclink_with_center_tap`: $2 \times 14.4\,\mathrm{mF}$ in series, $107\,\mathrm{k\Omega}$ across each capacitor; gives v_{dc} in pu of $u_{\mathrm{dc,nom}} = 1070\,\mathrm{V}$
`inv_psm_mod1/inverter`	`Three-Phase Bridge (ideal)`: six ideal switches with antiparallel diodes, $R_{\mathrm{on}} = 1\,\mathrm{m\Omega}$, gate threshold 2.5 V, gate order a+, a−, b+, b−, c+, c−; built-in passives off; the gate generator gives 2.94 µs of dead time; the `timer` (15 kHz, single update) triggers the control
`inv_psm_mod1/ inverter_output_filter`	three-phase inductor, $L_{\mathrm{Fi}} = 230\,\mu\mathrm{H}$, $R_{\mathrm{LFi}} = 1.8\,\mathrm{m\Omega}$
`psm`	library `permanent_magnet_synchronous_machine`, one system: $R_s = 30.1\,\mathrm{m\Omega}$, $L_d = 6.24\,\mathrm{mH}$, $L_q = 7.56\,\mathrm{mH}$, $\psi_{\mathrm{m}} = 3.774\,\mathrm{V\,s}$, 104 poles
`fixed_speed_setting`	ideal speed source: step to 17.8 rpm at 25 ms through a 0.1 s lag (energy production, default)
`motor_load_setting`	inertia and load torque, for the speed-control variant (not the default)

2.2. Control chain

The control is in the subsystem `inv_psm_mod1/inverter_control`; it is triggered every t_{s} by the timer and works in pu of the machine of one system:

1. **Acquisition.** Phase currents and DC voltage are quantized (2^{-11} pu) and delayed by one step (Unit Delay); encoder angle and speed are quantized (12 bit) and delayed by one step.
2. **Torque reference** (`torque_reference`): the speed of the control through a 1 Hz low-pass filter, the torque curve of the turbine, $-1/\tau_{\mathrm{bez}}$: the q current reference, with $i_d^* = 0$ (Section 5). Variant: speed PI (`use_speed_control`).
3. **Current reference** (`current_reference_generator`): i_q^* turned to $\alpha\beta$ with the control angle θ_{c}.
4. **Predictive control** (`inner_ctrl/mp control/mpc_pu_d`, MATLAB Function `mpc`, Section 4): chooses one of the eight states, writes the six gates and gives the observers the voltage of the vector applied before.
5. **Observers** (`bemf_observer1`): the EKF (default, Section 6) or one of two back-EMF observers (Simulink, C-Caller); the predictive control always takes flux and speed from the EKF block, whose speed output comes from the DIO on the EKF angle.
6. **Encoder–sensorless switch**: angle and speed for the torque curve and the current reference, from the encoder (through a DIO) or from the EKF block (`use_psm_encoder` = 0, sensorless, by default; the encoder is used while the speed is zero).
7. **Timing**: the local clock `global timing/time_inv1` has the gain 1.001 (open point 2); its alignment loop `phase_compensation_1` has zero gains.

2.3. Timing of one control step

Figure 2 shows what happens at step k. The current and the DC voltage sampled at t_{k-1} are used at step k. The vector $\underline{u}_k$ chosen at step k acts over $[t_k, t_{k+1})$, so it decides the current at t_{k+1}, while $\underline{u}_{k-1}$ is still acting when $\underline{u}_k$ is chosen: the control first computes the current of now with $\underline{u}_{k-1}$ (delay compensation, Section 4.2). The current measured at step k was made by $\underline{u}_{k-2}$, which is the voltage the EKF needs with that current (Section 6.5). The outputs of the EKF are read from its state memory: at step k they are the estimate of t_{k-2}.

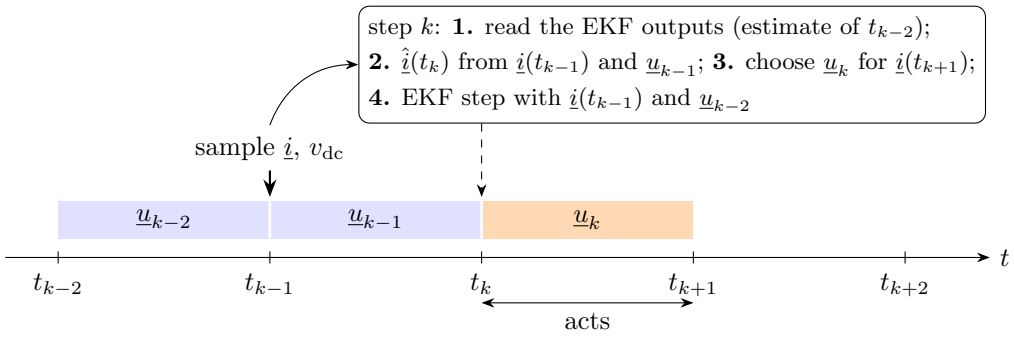


Figure 2: Timing of one control step: the samples taken at t_{k-1} are used at step k; the vector chosen at step k acts over the next step.

3. The machine seen from the bridge

3.1. Per-unit bases

The control works in the per unit of one system of the machine. The bases are those of `pmsm_setup` of the library, computed in `private/psm_machine_param.m`: the base speed is the rated speed, the base current the peak rated phase current, the base torque the rated torque, the base flux the magnet flux and the base voltage the back-EMF at rated speed. Table 3 gives the numbers. The DC voltage comes from the DC-link block in pu of $u_{\mathrm{dc,nom}} = 1070\,\mathrm{V}$, the base of the 690 V data set of the inverter; $k_u = u_{\mathrm{dc,nom}}/u_{\mathrm{bez}} = 2.925$ turns it into pu of u_{bez}.



Table 3: Bases and per-unit values of one system (`param.psm.machine`, `param.inv.hw`).

Quantity	Value	Formula, per unit
rated speed, pole pairs	17.8 rpm (1.864 rad/s), 52	$\omega_{\mathrm{m,bez}}$
ω_{bez}	96.93 rad/s (15.43 Hz)	$52\,\omega_{\mathrm{m,bez}}$; time base $1/\omega_{\mathrm{bez}} = 10.3$ ms
τ_{bez}	143.1 kN m	rated torque of the machine (858.4 kN m) / 6
i_{bez}	486.0 A peak	$\sqrt{2} \cdot 2062\,\mathrm{A}/6$
ψ_{m}	3.774 V s	$\tau_{\mathrm{bez}}/(\frac{3}{2}\,52\,i_{\mathrm{bez}})$, 1 pu
u_{bez}	365.8 V peak phase	$\psi_{\mathrm{m}}\,\omega_{\mathrm{bez}}$; power base $\frac{3}{2}u_{\mathrm{bez}}i_{\mathrm{bez}} = 266.7$ kW
X_{bez}, L_{bez}	0.7526 Ω, 7.765 mH	$u_{\mathrm{bez}}/i_{\mathrm{bez}}$, $X_{\mathrm{bez}}/\omega_{\mathrm{bez}}$
R_s, L_d, L_q	30.1 mΩ, 6.24 mH, 7.56 mH	0.0400, 0.8036, 0.9736
R_{LFi}, L_{Fi}	1.8 mΩ, 230 µH	0.0024, 0.0296
seen from the bridge	$R = R_s + R_{\mathrm{LFi}}$, $L_d + L_{\mathrm{Fi}}$, $L_q + L_{\mathrm{Fi}}$	0.0424, 0.8333, 1.0033
control step	$t_{\mathrm{s}} = 66.7$ µs	$h = t_{\mathrm{s}}\,\omega_{\mathrm{bez}} = 0.00646$
DC voltage	$u_{\mathrm{dc,nom}} = 1070$ V; battery 1249 V open circuit	$k_u = 2.925$; $v_{\mathrm{dc}} = 1.168$

3.2. Model in the rotor frame

With the output inductor in series with the machine, the bridge sees $L'_d = L_d + L_{\mathrm{Fi}}$, $L'_q = L_q + L_{\mathrm{Fi}}$ and $R = R_s + R_{\mathrm{LFi}}$. In the rotor frame and in pu (time $\tau = \omega_{\mathrm{bez}}t$, $\psi_{\mathrm{m}} = 1$):

$$L'_d \frac{\mathrm{d}i_d}{\mathrm{d}\tau} = u_d - R\,i_d + \omega L'_q i_q, \qquad L'_q \frac{\mathrm{d}i_q}{\mathrm{d}\tau} = u_q - R\,i_q - \omega L'_d i_d - \omega\,\psi_{\mathrm{m}}, \tag{3.1}$$

$$T = \psi_{\mathrm{m}}\,i_q + (L_d - L_q)\,i_d\,i_q. \tag{3.2}$$

The machine has $L_d < L_q$: a negative i_d would add reluctance torque, but the model runs with $i_d^* = 0$ (Section 5). The derivation of the PMSM model in machine variables, $\alpha\beta$ and dq is in [10].

3.3. Active flux: one inductance for the control

The predictive control and the EKF work in the stationary frame with one inductance L for both axes. Write the flux linked by the bridge (machine and inductor) as $L\,\underline{i}$ plus a rest $\underline{\psi}$. In the rotor frame the rest is

$$\underline{\psi} = \big(\psi_{\mathrm{m}} + (L'_d - L)\,i_d\big) + \mathrm{j}\,(L'_q - L)\,i_q. \tag{3.3}$$

With $L = L'_q = L_q + L_{\mathrm{Fi}}$ the rest is $\psi_{\mathrm{m}} + (L_d - L_q)\,i_d$ on the d axis for any current: the *active flux* [6]. Its angle is the rotor angle, and the stationary-frame model of the bridge, the inductor and the machine is

$$L\,\frac{\mathrm{d}\underline{i}}{\mathrm{d}\tau} = \underline{u} - R\,\underline{i} - \underline{e}, \qquad \underline{e} = \mathrm{j}\,\omega\,\underline{\psi}, \qquad \frac{\mathrm{d}\underline{\psi}}{\mathrm{d}\tau} = \mathrm{j}\,\omega\,\underline{\psi}, \qquad T = \psi_\alpha i_\beta - \psi_\beta i_\alpha, \tag{3.4}$$

exact while i_d is constant (a change of i_d changes the length of $\underline{\psi}$, a term the control neglects). The torque formula is exact for any current. This is why the model uses $L = L_q + L_{\mathrm{Fi}} = 1.0033$ pu



and $R = R_s + R_{\mathrm{LFi}} = 0.0424\,\mathrm{pu}$ (parameters `param.psm.ctrl.mpc.L_norm` and `R_norm`) in the predictive control, in the EKF and in the two back-EMF observers.

The same equation tells what happens when L is not the true L'_q: the observed flux (3.3) has a q component and its angle is wrong by

$$\Delta\theta = \arctan \frac{(L'_q - L)\, i_q}{\psi_\mathrm{m} + (L'_d - L)\, i_d} \approx (L'_q - L)\, i_q. \tag{3.5}$$

At the rated $i_q = -0.874\,\mathrm{pu}$ one percent of L_q moves the angle by about 0.49°. Before November 2026 the control used $L_s = (L_d + L_q)/2 = 0.889\,\mathrm{pu}$ of the machine alone, without the inductor: (3.5) gives −5.7° at rated torque. With $L = L'_q$ and a machine whose inductances are 20 % higher than the data, (3.5) gives −9.7°; the switched simulation gives −9.5° (Section 7).

4. Finite-set predictive current control

4.1. Voltage of a state

A state of the two-level bridge is $s = (s_a, s_b, s_c)$, $s_x \in \{0, 1\}$, with the leg voltages $s_x v_\mathrm{dc}$ to the negative rail. The space vector of the bridge voltage, in pu of u_bez, is

$$u_\alpha = k_u\, v_\mathrm{dc}\, \tfrac{2}{3}\big(s_a - \tfrac{1}{2}s_b - \tfrac{1}{2}s_c\big), \qquad u_\beta = k_u\, v_\mathrm{dc}\, \tfrac{1}{\sqrt{3}}\big(s_b - s_c\big), \tag{4.1}$$

with v_dc the measured DC voltage (pu of $u_\mathrm{dc,nom}$, one step late like the currents) and $k_u = 2.925$ (`u_scale`). The six active states have the length $\frac{2}{3}k_u v_\mathrm{dc}$: 2.28 pu at the open-circuit voltage of the battery (1249 V) and 2.32 at the 1274 V of rated operation. The two zero states [0 0 0] and [1 1 1] give no voltage. Figure 3 shows the eight states with the numbers of the table of the code. The largest sinusoidal voltage is the inscribed circle, $v_\mathrm{dc}/\sqrt{3}$ (1.97 pu at 1249 V); at rated speed the machine needs 1.30 pu (476 V peak), two thirds of it.

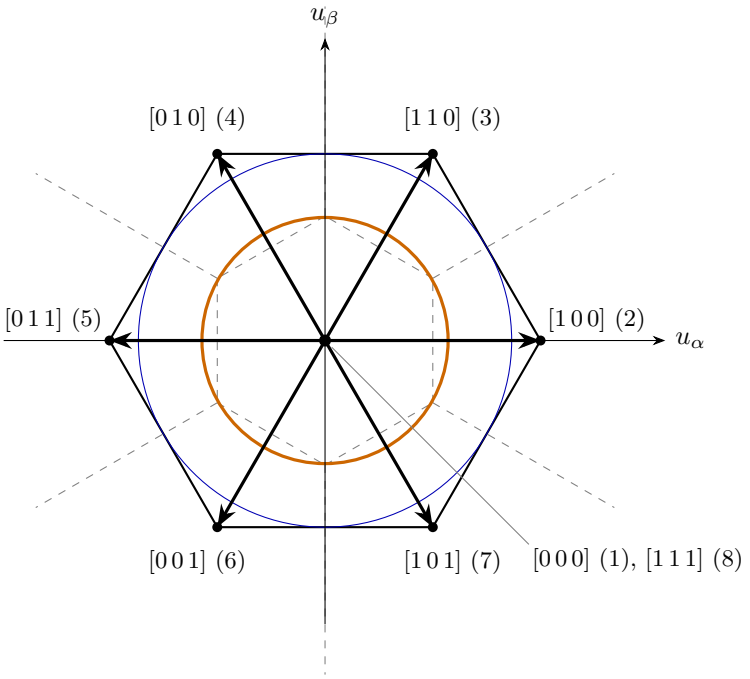


Figure 3: The eight states of the two-level bridge in the $\alpha\beta$ plane (pu of u_{bez} at 1249 V), with their number in the table of the code; the two zero states are at the origin. Blue: largest sinusoidal voltage (linear limit $v_{\text{dc}}/\sqrt{3}$, 1.97 pu); orange: voltage of the machine at rated speed (1.30 pu). Dashed: the regions of the voltages nearest to each state, which the cost function picks (Section 4.3).

4.2. Prediction and delay compensation

The prediction is the forward Euler step of (3.4) over one control step, with the back-EMF $\underline{e} = \mathrm{j}\hat{\omega}\underline{\hat{\psi}}$ of the EKF:

$$\underline{i}(t_{k+1}) = \underline{i}(t_k) + \hat{\gamma}\,\big(\underline{u}_k - R\,\underline{i}(t_k) - \underline{e}\big), \qquad \hat{\gamma} = \frac{h}{L} = \frac{t_{\text{s}}\,\omega_{\text{bez}}}{L} = 0.00644. \tag{4.2}$$

The current of now, $\underline{i}(t_k)$, is not measured: the measurement is $\underline{i}(t_{k-1})$. So the function first moves the measured current one step forward with the vector that is acting, $\underline{u}_{k-1}$ (delay compensation [5]):

$$\hat{\underline{i}}(t_k) = \underline{i}(t_{k-1}) + \hat{\gamma}\,\big(\underline{u}_{k-1} - R\,\underline{i}(t_{k-1}) - \underline{e}\big), \tag{4.3}$$

and then predicts, for each of the eight states, $\underline{i}_s(t_{k+1})$ from $\hat{\underline{i}}(t_k)$ with (4.2) and the voltage (4.1) of the state. The voltage of the state uses the DC voltage measured at t_{k-1}: the DC link (7.2 mF across the battery) moves by a few volts at most in one step.

4.3. Cost function and switching term

The cost of a state is the squared current error at t_{k+1} plus a switching term:

$$J(s) = k_{\text{i}}\,\big|\underline{i}^* - \underline{i}_s(t_{k+1})\big|^2 + k_{\text{sw}}\,N_{\text{sw}}(s), \qquad N_{\text{sw}}(s) = \sum_{x=a,b,c} |s_x - s_x^-|, \tag{4.4}$$



with s^- the state applied in the last step, $k_\mathrm{i} = 1$ (`k_kalman`) and $k_\mathrm{sw} = 10^{-5}$ (`k_sw`); the first state with the lowest cost wins. Since the predicted current is affine in the voltage with the scalar coefficient $\hat{\gamma}$, the current error is $\hat{\gamma}$ times a voltage distance:

$$\left|\underline{i}^* - \underline{i}_s(t_{k+1})\right| = \hat{\gamma}\left|\underline{u}^* - \underline{u}_s\right|, \qquad \underline{u}^* = \frac{\underline{i}^* - \hat{\underline{i}}(t_k)}{\hat{\gamma}} + R\,\hat{\underline{i}}(t_k) + \underline{e}, \tag{4.5}$$

where $\underline{u}^*$ is the voltage that would bring the current exactly onto the reference (the deadbeat voltage). Without the switching term the control applies the state whose voltage is nearest to $\underline{u}^*$: the regions of Figure 3. The region of the zero states reaches $\frac{2}{3\sqrt{3}} k_u v_\mathrm{dc} = 1.31\,\mathrm{pu}$ at its corners, about the rated voltage (1.30 pu): at rated speed the zero states are applied in 39 % of the steps.

The two zero states have the same voltage, and other states are often almost as good as the best one. The switching term decides these ties: one leg that changes state costs as much as a current error of $\sqrt{k_\mathrm{sw}/k_\mathrm{i}} = 0.0032\,\mathrm{pu}$ (1.5 A), about half of the rms current error at rated power. At rated power it lowers the switching from 3.5 to 2.8 kHz per device with the same current error (2.8 A rms), and it also makes the EKF angle quieter (std 0.35° instead of 3.0°, Table 7). Ten times more ($k_\mathrm{sw} = 10^{-4}$) gives 40 % less switching for 25 % more current error; 10^{-3} is too much (11.6 A rms).

4.4. The current loop as a deadbeat loop

Without the limit of the finite set, the choice (4.5) is a deadbeat controller. On one axis, with R and $\underline{e}$ right, let b be the true current change for a unit voltage over one step and $\hat{\gamma}$ the one of the prediction (including any error of the voltage base); call $g = b/\hat{\gamma}$ the gain of the loop and $\varepsilon = i^* - i$ the error. Since the true current obeys $i(t_k) = i(t_{k-1}) + b\,u_{k-1}$ (plus the terms that the prediction knows),

$$\varepsilon_{k+1} = (1-g)\,\varepsilon_{k-1}, \qquad z^2 = 1 - g, \tag{4.6}$$

deadbeat for $g = 1$ and stable for $0 < g < 2$. The prediction uses $L = L'_q$ on both axes, so $g = 1.000$ on the q axis and $g = L'_q/L'_d = 1.204$ on the d axis: there the error is multiplied by -0.204 every two steps (poles $|z| = 0.45$).

Without the delay compensation the prediction starts from the measured current as if it were the current of now, $u_k = (i^* - i_{k-1})/\hat{\gamma}$, and

$$\varepsilon_{k+1} = \varepsilon_k - g\,\varepsilon_{k-1}, \qquad z^2 - z + g = 0, \qquad |z| = \sqrt{g}. \tag{4.7}$$

Even with $g = 1$ the poles are on the unit circle: an undamped oscillation at $f_\mathrm{s}/6 = 2.5\,\mathrm{kHz}$. Table 4 compares the loop of the model before November 2026 (no delay compensation, states computed in pu of $u_\mathrm{dc,nom}$ with $v_\mathrm{dc} = 1$, so that the prediction saw the voltage 3.42 times too small, and L_s in place of L) with the fixed loop.



Table 4: Current loop seen as a deadbeat loop on one axis (no vector limit).

Loop	g	**poles** $\lvert z\rvert$	**behaviour**
before the fix, q axis	3.03	1.74	unstable, oscillation at 3.05 kHz
before the fix, d axis	3.64	1.91	unstable, oscillation at 3.12 kHz
before the fix, q axis, bridge inductors 1 mH on	2.68	1.64	unstable, oscillation at 3.01 kHz
no delay compensation, $g = 1$	1.00	1.00	undamped, 2.5 kHz
fixed, q axis	1.000	0	deadbeat
fixed, d axis	1.204	0.45	error $\times(-0.204)$ every two steps

The finite set keeps the current bounded even when the linear loop is unstable, but the old control applied vectors that were far too large (Figure 6, top). With the fixed loop the current follows the reference within the ripple of the finite set: 2.8 A rms at rated power.

4.5. Gates and the voltage for the observers

The chosen state gives the six gate commands (0 or 15 V) of a+, a−, b+, b−, c+, c−; the gate generator of the inverter adds the dead time. The outputs `ualpha`, `ubeta` of the function are the voltage $\underline{u}_{k-1}$ of the vector applied in the last step, in pu of u_{bez}; the Unit Delay after the block makes it $\underline{u}_{k-2}$, the vector that made the current measured at step k, which is what the EKF needs (Section 6.5). The gains between the function and the observers are 1 (`Gain7`, `Gain1`, `u_psm_scale`, `u_psm_scale_ekf`). The dead time is not compensated: during 2.94 µs a leg keeps its old level when the current needs the switch that turns on, a small voltage error that the check shows to be harmless (case “no dead time” of Table 7). The whole function is listed in Appendix A.

5. Torque and current reference

The energy-production mode (`use_torque_curve` = 1, default) takes the torque reference from the torque curve of the turbine (Figure 4): the speed of the control (absolute value, pu) goes through a first-order low-pass filter at 1 Hz,

$$y_k = (1 - g_0)\, y_{k-1} + g_0\, \omega_{\mathrm{c}}, \qquad g_0 = 2\pi \cdot 1\,\mathrm{Hz} \cdot t_{\mathrm{s}} = 4.19 \cdot 10^{-4}, \tag{5.1}$$

then $\times$ 17.8 rpm, the lookup table of the torque curve (linear, with extrapolation), $\times(-1/\tau_{\mathrm{bez}})$ and a saturation at zero. The result is the q current reference i_q^* (negative: generator), with $i_d^* = 0$. The block `current_reference_generator` turns it into $\alpha\beta$ with the control angle θ_{c}:

$$i_\alpha^* = -i_q^* \sin\theta_{\mathrm{c}}, \qquad i_\beta^* = i_q^* \cos\theta_{\mathrm{c}}. \tag{5.2}$$

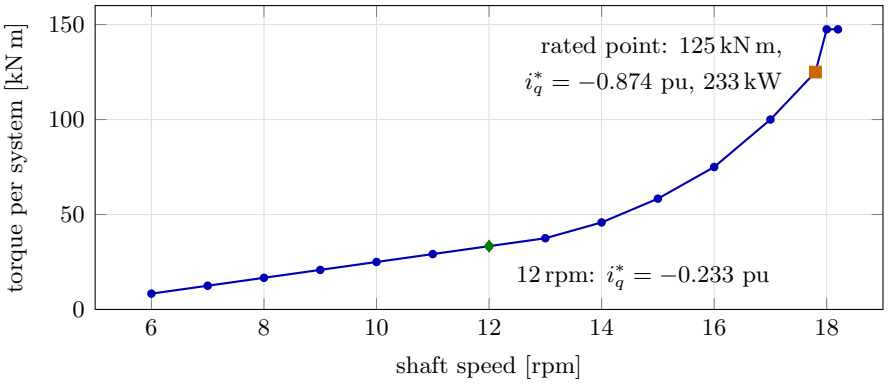


Figure 4: Torque curve of the model per system (`param.load.rpm_base`, `torque_base`: the curve of the whole machine divided by six).

At the rated speed the curve gives 125 kN m per system: $i_q^* = -0.874$ pu, 233 kW, 300 A rms, a bridge voltage of 476 V peak (1.30 pu) and $\cos\varphi = -0.74$ (power to the DC link). The curve is steep there: 31 kN m per rpm, the torque grows by 25 % from 17 to 17.8 rpm, so a speed estimate 0.1 % low gives about −0.45 % of torque. This is why the speed of the control matters (Section 6.6). The lookup table extrapolates the curve down to zero at 4 rpm; below that the saturation keeps the reference at zero, so the current starts at about 0.18 s in the default run.

The control angle is old when the vector acts: the EKF angle used at step k is the estimate of t_{k-2} and the vector acts until t_{k+1}, three steps, $3h = 1.11°$ at rated speed. The current is then about 1.1° behind the q axis, $i_d \approx -0.017$ pu at rated i_q; the model does not compensate it (open point 4).

The speed-control variant (`use_speed_control` = 1) replaces the torque curve with a speed PI ($k_p = 2.5$, $k_i = 18$/s, limit 1.4 pu) and the fixed speed source with `motor_load_setting`; it was not checked in November 2026.

6. Extended Kalman filter

6.1. State, model and output

The EKF of the block `EKF_bemf_observer/extended_kalman_observer` [7, 8] has the state and the output

$$\boldsymbol{x} = \begin{bmatrix} i_\alpha & i_\beta & \psi_\alpha & \psi_\beta & \omega & \tau_\mathrm{L} \end{bmatrix}^\mathsf{T}, \qquad \boldsymbol{y} = C\boldsymbol{x} = \begin{bmatrix} i_\alpha & i_\beta \end{bmatrix}^\mathsf{T}, \tag{6.1}$$

all in pu, and the model (3.4) with a mechanical part, written in per-unit time $\tau = \omega_\mathrm{bez} t$:

$$\frac{\mathrm{d}\boldsymbol{x}}{\mathrm{d}\tau} = f(\boldsymbol{x}, \underline{u}) = \begin{bmatrix} (u_\alpha - R\,i_\alpha + \omega\,\psi_\beta)/L \\ (u_\beta - R\,i_\beta - \omega\,\psi_\alpha)/L \\ -\omega\,\psi_\beta \\ \omega\,\psi_\alpha \\ (\psi_\alpha i_\beta - \psi_\beta i_\alpha - \tau_\mathrm{L})/j_\mathrm{m} \\ 0 \end{bmatrix}. \tag{6.2}$$



L and R are the values of the predictive control ($L_q + L_{\mathrm{Fi}}$, $R_s + R_{\mathrm{LFi}}$), so the flux is the active flux of Section 3.3, on the d axis for any i_d. The inertia of the model is $j_{\mathrm{m}} = 10^6$ (`jm_ekf`): the torque terms are negligible and the speed is a free state, moved only by the corrections of the filter (the shaft speed of the wind generator changes slowly). The outputs of the block are the flux $\hat{\psi}_\alpha$, $\hat{\psi}_\beta$ (to the predictive control), the angle $\hat{\theta} = \mathrm{atan2}(\hat{\psi}_\beta, \hat{\psi}_\alpha)$ and the speed of the DIO on that angle (Section 6.6).

6.2. Jacobian

The model has products of two states. The block writes its Jacobian $F_{\mathrm{c}}(\boldsymbol{x}) = \partial f / \partial \boldsymbol{x}$ as a sum of constant matrices weighted by the states,

$$F_{\mathrm{c}}(\boldsymbol{x}) = A_1 + A_2\, i_\alpha + A_3\, i_\beta + A_4\, \psi_\alpha + A_5\, \psi_\beta + A_6\, \omega, \tag{6.3}$$

with the input matrix B and the non-zero entries of Table 5 (parameters `A1_tilde_ekf` ... `A6_tilde_ekf`, `B_tilde_ekf`, built in `private/psm_ctrl_param.m` as in `ekf_pmsm_setup` of the library).

Table 5: Non-zero entries (row, column) of the coefficient matrices of the Jacobian; states in the order $i_\alpha, i_\beta, \psi_\alpha, \psi_\beta, \omega, \tau_{\mathrm{L}}$.

Matrix	**weighted by**	**entries**
A_1	1	$(1,1) = (2,2) = -R/L$; $(5,6) = -1/j_{\mathrm{m}}$
A_2	i_α	$(5,4) = -1/j_{\mathrm{m}}$
A_3	i_β	$(5,3) = 1/j_{\mathrm{m}}$
A_4	ψ_α	$(2,5) = -1/L$; $(4,5) = 1$; $(5,2) = 1/j_{\mathrm{m}}$
A_5	ψ_β	$(1,5) = 1/L$; $(3,5) = -1$; $(5,1) = -1/j_{\mathrm{m}}$
A_6	ω	$(1,4) = 1/L$; $(2,3) = -1/L$; $(3,4) = -1$; $(4,3) = 1$
B	input u_α, u_β	$(1,1) = (2,2) = 1/L$
C	output	$(1,1) = (2,2) = 1$

6.3. Prediction of the state and of the covariance

The block discretizes with the forward Euler step $h = t_{\mathrm{s}}\, \omega_{\mathrm{bez}}$. It builds the transition matrix twice, in two copies of the subsystem `A_ekf`:

$$\boldsymbol{x}^- = F_{\mathrm{pred}}\, \boldsymbol{x} + h\, B\, \underline{u}, \qquad F_{\mathrm{pred}} = I + h\left(A_1 + A_2 i_\alpha + A_3 i_\beta + A_4 \psi_\alpha + A_5 \psi_\beta\right), \tag{6.4}$$

$$P^- = F_{\mathrm{cov}}\, P\, F_{\mathrm{cov}}^{\mathsf{T}} + Q_{\mathrm{K}}, \qquad F_{\mathrm{cov}} = I + h\, F_{\mathrm{c}}(\boldsymbol{x}). \tag{6.5}$$

The covariance needs the full Jacobian F_{cov}. The state prediction must be $\boldsymbol{x} + h f(\boldsymbol{x}, \underline{u})$, the model itself, and the full Jacobian would not give it: for a product of two states the Jacobian holds the term twice ($\partial(\omega\psi)/\partial\omega \cdot \omega + \partial(\omega\psi)/\partial\psi \cdot \psi = 2\,\omega\psi$). The products $\omega\psi$ of the first four rows already come once through A_4 and A_5, so the copy of the state prediction reads `A6_pred_ekf` $= 0$ on its A_6 path and $F_{\mathrm{pred}}\boldsymbol{x} + hB\underline{u} = \boldsymbol{x} + hf(\boldsymbol{x}, \underline{u})$ in those rows. Before November 2026 that path had A_6 without the factor ω_{bez}: it added $1/\omega_{\mathrm{bez}}$ of a second $\omega\psi$, 1 % too much rotation



and back-EMF, and a speed estimate about 1 % low. In the speed row the torque products still come twice (from A_2, A_3 and from A_4, A_5); with $j_\mathrm{m} = 10^6$ this has no effect (open point 9).

6.4. Correction and noise

The measured current corrects the prediction:

$$K = P^- C^\mathsf{T} \big(C P^- C^\mathsf{T} + R_\mathrm{K}\big)^{-1}, \qquad \boldsymbol{x} = \boldsymbol{x}^- + K\left(\underline{i}_\mathrm{meas} - C\boldsymbol{x}^-\right), \qquad P = (I - KC)\,P^-, \quad (6.6)$$

with the noise covariances of `ekf_pmsm_setup`:

$$Q_\mathrm{K} = t_\mathrm{s}\, q_\mathrm{ekf}\ \mathrm{diag}\big(t_\mathrm{s}^2,\, t_\mathrm{s}^2,\, \tfrac{1}{2},\, \tfrac{1}{2},\, 1,\, 1\big), \qquad R_\mathrm{K} = t_\mathrm{s}\, r_\mathrm{ekf}\, I_2, \qquad q_\mathrm{ekf} = \tfrac{1}{100},\ r_\mathrm{ekf} = 1, \qquad (6.7)$$

with t_s in seconds. The gain does not change when Q_K, R_K and P are scaled together, so only $q_\mathrm{ekf}/r_\mathrm{ekf}$ matters. The current states have almost no process noise ($t_\mathrm{s}^2 = 4.4 \cdot 10^{-9}$): the filter trusts the current equation, i.e. the model of the inductor and of the bridge voltage, and lets the flux, the speed and the load torque move. The legacy value $q_\mathrm{ekf} = 1/10$ makes the filter follow the model error too fast at 15 kHz: with the machine inductances 10 % below the data the angle jumps (std 14°) and at −15 % it is lost. With 1/100 the angle is held from −25 % to +20 % and its noise at rated power is 0.35° std instead of 0.79° (Table 7).

6.5. Voltage and current given to the filter

At step k the filter corrects with $\underline{i}(t_{k-1})$, the same measured current as the predictive control, so its prediction from t_{k-2} to t_{k-1} needs the voltage that acted over $[t_{k-2}, t_{k-1})$: $\underline{u}_{k-2}$ (Figure 2). The predictive control outputs $\underline{u}_{k-1}$ and the Unit Delay of `mp control` adds the second step (Section 4.5). The voltage is the one the predictive control computed with (4.1), in pu of u_bez, from the measured DC voltage; the dead time is not in it. Before November 2026 the filter received 0.74 times the true voltage, of the vector chosen in the same step.

6.6. Angle, flux and speed: the double integrator observer

The speed of the EKF block is not the speed state of the filter: it comes from a DIO on the EKF angle (block `extended_kalman_observer/double_integrator_observer`), the same observer that gives angle and speed from the encoder. With the angle error $\varepsilon = \mathrm{wrap}(\hat{\theta}_\mathrm{EKF} - \theta_\mathrm{o})$ it updates, every step,

$$\theta_\mathrm{o} \leftarrow \mathrm{wrap}(\theta_\mathrm{o}) + t_\mathrm{s}\left(\omega_\mathrm{bez}\,\omega_\mathrm{o} + k_\theta\,\varepsilon\right), \qquad \omega_\mathrm{o} \leftarrow \omega_\mathrm{o} + t_\mathrm{s}\,k_\omega\,\varepsilon/\omega_\mathrm{bez}, \qquad (6.8)$$

with $k_\theta = 479.4/\mathrm{s}$ and $k_\omega = 36\,980/\mathrm{s}^2$ placed for the poles $-\omega_\mathrm{bez}$ and $-4\,\omega_\mathrm{bez}$ (15.4 and 61.7 Hz, mapped with $e^{p\,t_\mathrm{s}}$; close to $k_\theta = 5\omega_\mathrm{bez}$, $k_\omega = 4\omega_\mathrm{bez}^2$ of the continuous loop $s^2 + k_\theta s + k_\omega$). The loop has two integrators, so it follows an angle that turns at constant speed with no error, and its speed is the true speed of the angle. Its states start at $\theta_\mathrm{o} = \pi$, $\omega_\mathrm{o} = 0$ (open point 8).



Why the speed state is low. The forward Euler step turns the flux by $\underline{\psi}\,(1+\mathrm{j}\omega h)$, which also makes it longer by the factor $\sqrt{1+(\omega h)^2}$, i.e. by $(\omega h)^2/2 = 2.1\cdot 10^{-5}$ every step at rated speed ($\omega h = 0.006\,46\,\mathrm{rad} = 0.37°$). The measured current pins the back-EMF $\omega\,|\underline{\psi}|$ and the angle, so the filter must take this growth back at every step through its correction, and it settles with a small offset shared between the flux and the speed: the speed state is low and the flux is long, while the angle turns at the right speed. An isolated test of the filter on exact data of its own model (companion script `py/ekf_speed_bias.py`, Table 6) gives $-0.28\,\%$ of speed at rated speed and $-0.17\,\%$ at half speed, the same with and without load and almost the same with $q_{\mathrm{ekf}} = 1/10$; the clock gain 1.001 adds $-0.10\,\%$. A prediction exact in the rotation (matrix exponential or second-order Taylor step) has no offset; the model keeps the Euler step and takes the speed from the DIO, which gives 0.9992 of the true speed in the switched simulation, i.e. only the error of the clock. With the speed state the torque would be $1.2\,\%$ lower on the steep torque curve (case "speed from the EKF state" of Table 7).

Table 6: Isolated test of the EKF at rated speed (`py/ekf_speed_bias.py`): exact plant, deadbeat voltage, no switching.

Case	speed state / true speed	angle error	$\lvert\hat{\underline{\psi}}\rvert$
forward Euler (the block), $q_{\mathrm{ekf}} = 1/100$	0.99717	$+0.11°$	1.0039
forward Euler, $q_{\mathrm{ekf}} = 1/10$	0.99725	$+0.14°$	1.0034
forward Euler, no load ($i_q = 0$)	0.99717	$+0.12°$	1.0039
forward Euler, half speed	0.99831	$+0.06°$	1.0020
forward Euler, clock 1.001	0.99618	$+0.06°$	1.0049
exact prediction (matrix exponential)	1.00000	$0.00°$	1.0000
second-order Taylor prediction	0.99999	$0.00°$	1.0000

6.7. Sensitivity to the machine inductances

The filter and the predictive control use $L = L_q + L_{\mathrm{Fi}}$ of the data. When the machine inductances differ, the observed flux leaves the d axis by (3.5): in the switched simulation the angle error is $+4.6°$ at $-10\,\%$, $+11.8°$ at $-25\,\%$ (torque $-6.3\,\%$) and $-9.5°$ at $+20\,\%$, about $0.47°$ per percent, and the angle is never lost in this range with $q_{\mathrm{ekf}} = 1/100$. With the encoder all cases are right (open point 3).

7. Design check

7.1. How the check works

The companion script `py/design_check_mpc_psm.py` (Python, numpy) simulates the whole chain at vector level for 1.25 s, the stop time of the model, with the numbers of `init_model.m`:

- shaft speed of `fixed_speed_setting`, torque curve with the 1 Hz filter and the saturation, current reference on the q axis at the control angle;
- currents and DC voltage sampled, quantized (2^{-11} pu) and one step late; control clock 1.001; encoder–sensorless switch;



- the predictive control as a line-by-line copy of the function `mpc` (`py/mpc_port_2l.py`, checked against an Octave run of the MATLAB code for 2000 steps), and of the old function for the cases before the fix;
- the EKF rebuilt block by block, with the DIO;
- plant: the machine in the rotor frame (L_d, L_q plus the inductor), the two-level bridge with ideal switches and the 2.94 µs dead time (a leg keeps its old level when the current needs the switch that turns on), the DC link with its series resistance, the battery as open-circuit voltage, R_0 and $R_1 \parallel C_1$.

It does not model the gate filter and the Simscape solver. The values of Table 7 are averages over 1.0–1.25 s; the full output is `figures/design_check_mpc_psm_output.txt`.

7.2. Results

Table 7: Results of the switched simulation (17.8 rpm unless stated, averages over 1.0–1.25 s). Angle: error of the control angle (std); speed: speed of the control over the true speed; torque: against the torque curve at the true speed; switching: leg changes per second / 2.

Case	angle (std)	speed	torque	current error	switching
before the fix (`dev` model before PR #33, bridge passives on)	+9.4° (12.6°)	1.304	+16.0 %	50.4 A	959 Hz
before the fix, bridge passives off	lost	4.89	–	200 A	297 Hz
fixed, sensorless	−0.1° (0.35°)	0.9992	−1.7 %	2.8 A	2823 Hz
fixed, encoder	−0.4° (0.00°)	0.9990	−1.1 %	2.8 A	2863 Hz
fixed, no dead time	−0.7° (0.02°)	0.9990	−1.6 %	2.8 A	2802 Hz
fixed, 12 rpm	−0.3° (0.40°)	0.9989	−0.3 %	2.1 A	2158 Hz
fixed, bridge passives still on	−6.3° (0.30°)	0.9988	−1.0 %	2.6 A	2907 Hz
fixed, machine L +20 %	−9.5° (0.29°)	0.9987	−1.1 %	2.5 A	2919 Hz
fixed, machine L −10 %	+4.6° (0.50°)	0.9991	−3.0 %	3.0 A	2765 Hz
fixed, machine L −20 %	+9.4° (1.29°)	0.9993	−5.0 %	3.4 A	2641 Hz
fixed, machine L −25 %	+11.8° (2.07°)	0.9989	−6.3 %	3.7 A	2553 Hz
fixed, machine L −25 %, encoder	−0.4° (0.00°)	0.9995	−1.1 %	3.7 A	2519 Hz
fixed, legacy noise $q_{\mathrm{ekf}} = 1/10$	−0.2° (0.79°)	0.9987	−1.7 %	2.8 A	2806 Hz
fixed, $q_{\mathrm{ekf}} = 1/10$, machine L −10 %	+7.4° (13.9°)	1.0005	−4.7 %	15.5 A	2020 Hz
fixed, $q_{\mathrm{ekf}} = 1/10$, machine L −15 %	lost	0.998	–	142 A	600 Hz
fixed, no switching term ($k_{\mathrm{sw}} = 0$)	−1.2° (3.00°)	0.9999	−1.7 %	2.8 A	3503 Hz
fixed, $k_{\mathrm{sw}} = 10^{-4}$	−0.5° (0.25°)	0.9991	−1.5 %	3.5 A	1688 Hz
fixed, $k_{\mathrm{sw}} = 10^{-3}$	−0.7° (0.29°)	0.9991	−1.3 %	11.6 A	367 Hz
fixed, speed from the EKF state	−0.1° (0.29°)	0.9960	−2.9 %	2.8 A	2836 Hz

About −1.3 % of the torque error of every fixed case is the start of the run, not the control: the speed of the torque curve goes through the 1 Hz filter after the 0.1 s lag of the speed step, and at 1.0–1.25 s the filter is still 0.05 rpm behind on the steep part of the curve. The rest is the speed estimate (0.08 % low, −0.35 % of torque). With the fixed control the DC link sits at about 1274 V and the battery takes 219 kW; the zero states are applied in 39 % of the steps.

Figure 5 shows the start-up. Before the fix the control follows its reference at low speed, but

an oscillation of the angle and of i_d grows with the speed; at about 0.9 s (near rated speed) the EKF speed jumps to 1.3 pu and the torque reference goes to the top of the curve. The fixed control, sensorless or with the encoder, stays on the curve; the angle error is within 1.4° from the moment the torque reference leaves zero. Figure 6 shows the current over 100 ms: before the fix the current moves in large steps behind its reference, the fixed control follows the sinusoidal reference within the ripple of the finite set.

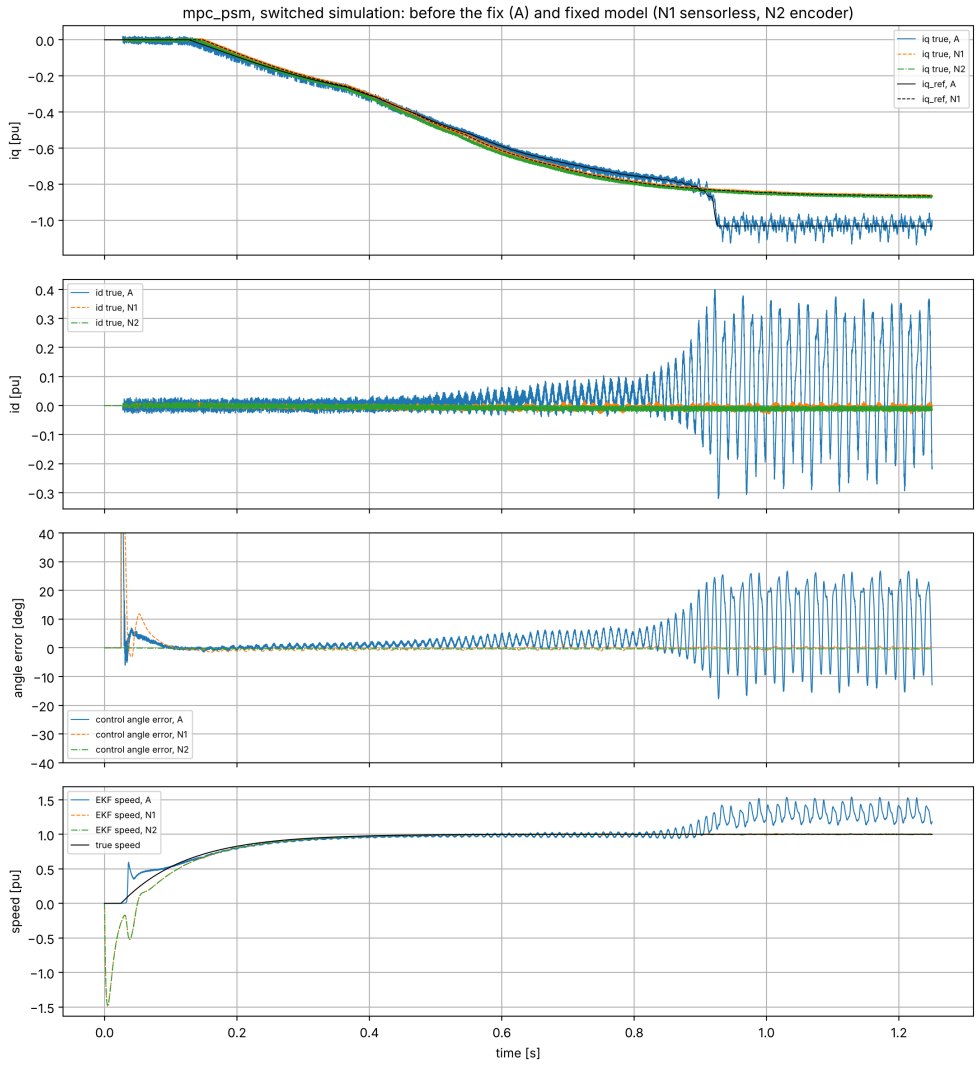


Figure 5: Start-up and rated operation in the switched simulation: q and d current, error of the control angle and speed of the control; model before the fix (A), fixed model sensorless (N1) and with the encoder (N2).

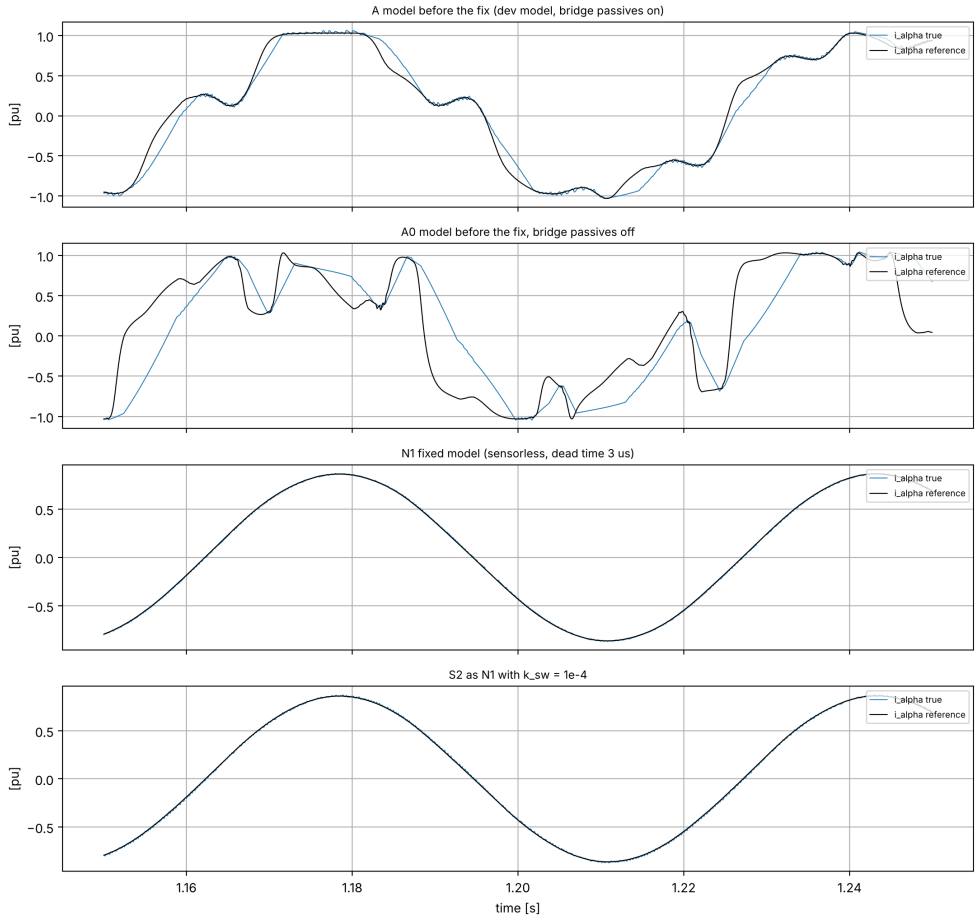


Figure 6: Current i_α and its reference over 100 ms at rated speed: model before the fix with and without the bridge passives (A, A0), fixed model (N1) and fixed model with $k_{\mathrm{sw}} = 10^{-4}$ (S2).

The check predicts for the scopes of the model: the current on its reference within about 3 A rms at rated power, the torque on the torque curve within the settling of the 1 Hz speed filter, the EKF angle within about 0.5° of the encoder angle and the DC link at about 1274 V.

8. What changed in November 2026

The model was migrated to the parameter structure `param` of the repository and its control was fixed in the same pull request (modelization-and-control PR #33, merged into `dev` on 5 November 2026). Table 8 lists the fixes; the README of the model gives the details of the migration (301 parameters re-pointed, 28 blocks without their Goto commented out, old and new names).



Table 8: Control fixes of November 2026.

#	Before	Now
1	states computed from the constant $v_{\mathrm{dc}} = 1$ in pu of $u_{\mathrm{dc,nom}}$, in a model in pu of u_{bez}: the prediction saw the voltage 3.42 times too small	measured v_{dc} times $k_u = 2.925$ (Section 4.1)
2	no delay compensation: unstable linear loop ($g = 3.0$ on q, 3.6 on d)	delay compensation, deadbeat on q (Section 4.4)
3	7 vectors, one zero vector, only neighbours after an active vector; switching term on the squared voltage with weight 0	all 8 states; $k_{\mathrm{sw}} = 10^{-5}$ times the legs that change state (Section 4.3)
4	prediction with $L_s = (L_d + L_q)/2 = 0.889\,\mathrm{pu}$ and R_s, without the inductor	$L = L_q + L_{\mathrm{Fi}} = 1.0033\,\mathrm{pu}$, $R = R_s + R_{\mathrm{LFi}}$ (Section 3.3)
5	EKF voltage 0.74 times the true one, of the vector chosen in the same step	$\underline{u}_{k-2}$ in pu of u_{bez}, gains 1 (Section 6.5)
6	EKF with L_s, R_s: $-5.7°$ of angle at rated torque	L and R of fix 4: active flux
7	A_6 path of the state prediction without ω_{bez}: speed about 1 % low	`A6_pred_ekf` = 0 (Section 6.3)
8	speed output = speed state of the EKF, 0.4 % low	speed from the DIO on the EKF angle (Section 6.6)
9	EKF noise $q_{\mathrm{ekf}} = 1/10$: angle lost at -15 % of the inductances	$q_{\mathrm{ekf}} = 1/100$
10	built-in passives of the new bridge block on: 2 mF starting at 0 V against the battery, 1 mH per phase unknown to the control	passives off
11	the two back-EMF observers got 0.86 times the voltage, one step too new, with L_s, R_s	same voltage, L and R as the EKF (not simulated)

9. Parameters

All numbers are in the structure `param` built by `init_model.m`; Table 9 lists those of the control. The README of the model describes every group of the structure.



Table 9: Control parameters of `mpc_psm` (`init_model.m`, `private/psm_ctrl_param.m`).

Name	Value	Meaning
`param.time.fpwm`, `trgo`	15 kHz, 0	control timer, single update: $t_s = 66.7\,\mu s$ (`param.glb_time.ts_inv`)
`param.time.dead_time`	3 µs	dead time of the gate generator (2.94 µs at the bridge)
`param.inv.behav.time_gain_module_2`	1.001	gain of the control clock `time_inv1`
`param.psm.ctrl.mpc.u_scale`	2.9252	$k_u = u_{dc,nom}/u_{bez}$
`param.psm.ctrl.mpc.L_norm`, `R_norm`	1.0033, 0.0424	$L = L_q + L_{Fi}$, $R = R_s + R_{LFi}$ (MPC, EKF, back-EMF observers)
`param.psm.ctrl.mpc.k_kalman`, `k_sw`	1, 10^{-5}	weights of the cost (4.4)
`param.psm.ctrl.ekf.Qkalman`	(6.7)	$q_{ekf} = 1/100$, diagonal $[t_s^2\ t_s^2\ \frac{1}{2}\ \frac{1}{2}\ 1\ 1]$
`param.psm.ctrl.ekf.Rkalman`	$t_s I_2$	$r_{ekf} = 1$
`param.psm.ctrl.ekf.A1_tilde_ekf` ... `A6_tilde_ekf`, `Bd_ekf`, `C_ekf`	Table 5	EKF model with $j_m = 10^6$ (`jm_ekf`); `Bd_ekf` $= t_s B$
`param.psm.ctrl.ekf.A6_pred_ekf`	0	A_6 path of the state prediction
`param.psm.ctrl.di_obs.ktheta`, `komega`	479.4, 36980	DIO gains, poles -1 and $-4\,\omega_{bez}$
`param.psm.ctrl.kp_w`, `ki_w`, `iq_lim`	2.5, 18/s, 1.4	speed PI (variant `use_speed_control`)
`param.psm.ctrl.emf_fb_p`, `emf_p`, `omega_th`	0.2, 0.08, 0	back-EMF observers (variants), threshold of the encoder–sensorless switch
`param.psm.u_psm_scale`, `u_psm_scale_ekf`	1, 1	voltage scale of the observers (the voltage is already in pu of u_{bez})
`param.psm.mode.use_torque_curve`, `use_psm_encoder`	1, 0	torque curve, sensorless
`param.psm.impl.use_ekf_bemf_module_1`	1	EKF active (the two back-EMF observers are variants)
`param.load.rpm_base`, `torque_base`	Figure 4	torque curve per system; `torque_overload_factor` = 1
`param.ctrl.filters.inv.g0_1Hz`, `g1_1Hz`	$4.19 \cdot 10^{-4}$	1 Hz speed filter of the torque curve
`param.meas.adc_quantization`	2^{-11}	quantization of the acquisitions (pu)
`param.ctrl.kp_align`, `ki_align`	0.25, 18/s	alignment loop of the clock (its mask has gains 0, open point 2)
`param.sim.simlength`, `nonlinear_iterations`	1.25 s, 5	stop time, Simscape nonlinear iterations

10. Files and how to run the model

How to run. (1) Clone the `library` repository next to `modelization-and-control` (or set the environment variable `PWR_CONTROL_LIBRARY`): the function `library_setup` of the model puts it on the MATLAB path and clones it when it is missing. (2) From the folder `foundations_of_power_electronics/mpc_psm` run `init_model.m`: it fills `param`, prints the bases and the design figures, opens `mpc_psm.slx`, sets the mechanical setting and checks that every `param.*` name read by the model exists and that every active From has its Goto. (3) Run the simulation (1.25 s).



Table 10: Files of the model and of this note.

File	Content
model `foundations_of_power_electronics/mpc_psm/` **of** `pwr-control/modelization-and-control` **(branch** `dev`**)**	
`init_model.m`	initialization as a plain-text live script: every number in `param`, design figures, model checks
`mpc_psm.slx`	the model: power stage, machine, control, logging
`mpc_psm_private.m`, `private/`	handles to the parameter functions (`timing_param`, `inv_hw_param`, `psm_machine_param`, `psm_ctrl_param`, `battery_param`, `library_setup`, the two model checks)
`README.md`	model, parameters, control fixes, migration, checks, design figures, open points
`spectrum.m`	FFT of a phase current of the machine and of a phase voltage
this note, `repos_doc/psm_mpc_ekf/` **of** `pwr-control/repositories-documentation`	
`psm_mpc_ekf.tex`, `.pdf`	source and PDF
`py/design_check_mpc_psm.py`	switched simulation of the whole chain (Section 7): writes `figures/design_check_mpc_psm_output.txt` and the two PNG figures; run `python3 design_check_mpc_psm.py ../figures` from `py/` (about two minutes)
`py/mpc_port_2l.py`	Python copies of the function `mpc` (fixed and old), used by the check
`py/ekf_speed_bias.py`	isolated test of the EKF (Table 6); its output is `figures/ekf_speed_bias_output.txt`
`figures/`	the outputs of the two scripts

11. Open points

1. **Check in MATLAB.** The fixes were designed with the switched simulation and the init was checked in Octave; a run of the model in MATLAB should show the scopes listed at the end of Section 7.
2. **Clock.** The control clock `time_inv1` has the gain 1.001 (15 015 Hz) while the constants use 66.7 µs: the prediction step is 0.1 % short and the speed is 0.999. Harmless here (no carrier), but it is the last error of the speed. The alignment loop `phase_compensation_1/alignment_ctrl` feeds this clock but has $k_p = k_i = 0$ typed in its mask.
3. **Sensorless range.** The angle error grows by about 0.47° per percent of error of the machine inductances (Section 6); the angle is held from −25 % to +20 %. If the real inductances are not known within about ±10 %, an adaptation of the inductance or of the angle is needed, or the encoder.
4. **Age of the control angle.** Three steps, 1.1° at rated speed, $i_d \approx -0.017$ pu (Section 5). An angle advance of $3\,\omega h$ in the current reference would remove it.
5. **DC voltage.** The battery holds the DC link at 1249–1274 V, 1.17–1.19 times $u_{\mathrm{dc,nom}}$ of the 690 V data set: the voltage rating of the switches should be checked. The control uses the measured DC voltage and follows the battery.
6. **Battery capacity.** The init uses $P/V = 208$ A h (1C) as an assumption; it does not



matter in 1.25 s.

7. **Simulation length.** 1.25 s ends while the 1 Hz speed filter is still settling (−1.3 % of torque); 2 s would show the steady state.
8. **Start of the speed estimate.** The DIO of the EKF block starts at $\theta_o = \pi$ while the EKF angle starts elsewhere: at standstill its speed swings down to about −1.5 pu (at 5 ms, Figure 5) and follows the true speed from about 0.15 s. The flux of the EKF is about zero at standstill and the torque reference is zero below 4 rpm, so the current is not disturbed, but the scopes show it.
9. **Torque row of the state prediction.** With `A6_pred_ekf` = 0 the speed row of F_{pred} counts the torque products twice (Section 6.3). It has no effect with $j_m = 10^6$; with a real inertia in the EKF model one of the two pairs (A_2, A_3 or the $1/j_m$ entries of A_4, A_5) should be left out of the prediction copy.
10. **Not checked.** The speed-control variant and the two back-EMF observers (fixed by construction, not simulated); the post-processing scripts `plotting_results_inv.m` and `power_losses_calculus.m` of the model folder come from other models and do not run on this one.

References

[1] pwr-control, *modelization-and-control* repository, model `foundations_of_power_electronics/mpc_psm` (branch `dev`) and its README, https://github.com/pwr-control/modelization-and-control.

[2] pwr-control, *library* repository (bridge, DC link, battery and machine blocks; `ekf_pmsm_setup`, `ctrl_pmsm_setup`), https://github.com/pwr-control/library.

[3] S. Kouro, P. Cortés, R. Vargas, U. Ammann, and J. Rodríguez, "Model predictive control—a simple and powerful method to control power converters," *IEEE Trans. Ind. Electron.*, vol. 56, no. 6, pp. 1826–1838, June 2009.

[4] J. Rodríguez and P. Cortés, *Predictive Control of Power Converters and Electrical Drives*. Wiley–IEEE Press, 2012.

[5] P. Cortés, J. Rodríguez, C. Silva, and A. Flores, "Delay compensation in model predictive current control of a three-phase inverter," *IEEE Trans. Ind. Electron.*, vol. 59, no. 2, pp. 1323–1325, Feb. 2012.

[6] I. Boldea, M. C. Paicu, and G.-D. Andreescu, "Active flux concept for motion-sensorless unified AC drives," *IEEE Trans. Power Electron.*, vol. 23, no. 5, pp. 2612–2618, Sept. 2008.

[7] S. Bolognani, L. Tubiana, and M. Zigliotto, "Extended Kalman filter tuning in sensorless PMSM drives," *IEEE Trans. Ind. Appl.*, vol. 39, no. 6, pp. 1741–1747, Nov./Dec. 2003.

[8] D. Simon, *Optimal State Estimation: Kalman, H_∞, and Nonlinear Approaches*. Wiley, 2006.



[9] D. Bagnara, *Finite-set predictive current control of a T-type wind-generator inverter: analysis and fixes of the model* ***mpc_ttype_psm***, pwr-control technical note, Rev. 00, November 2026, `research/power_electronics/fcs_mpc_ttype_psm` of https://github.com/pwr-control/repositories-documentation.

[10] D. Bagnara, *Permanent Magnet Synchronous Machines: model derivation, control and Simscape implementation*, pwr-control report, May 2021, `repos_doc/psm_model_observer_ctrl` of https://github.com/pwr-control/repositories-documentation.



A. The function mpc of the model

The MATLAB Function mpc of the block inner_ctrl/mp control/mpc_pu_d of mpc_psm.slx (chart 58), as it is in the model since November 2026. The inputs come from Constants that read param.psm.ctrl.mpc.*, param.psm.machine.omega_bez and param.glb_time.ts_inv; vdc comes from the From udcd_pu_inv_mod1.

```
function [ualpha, ubeta, cmg_g1, cmg_g2, cmg_g3, cmg_g4, cmg_g5, cmg_g6] = mpc(vdc, ialpha_ref, ...
    ibeta_ref, ialpha, ibeta, omega_hat, psi_alpha_hat, psi_beta_hat, u_scale, ...
    L_norm, omega_bez, R_norm, k_kalman, k_sw, ts_inv)
% Finite-set predictive current control of the two-level bridge: every control step it tests the 8 states
% of the bridge and applies the one with the lowest cost (current error and switching).
%
% Inputs, all per unit of the machine:
%   vdc                     DC-link voltage measured one control step before, pu of udc_nom
%   ialpha_ref, ibeta_ref   current reference, pu of ibez
%   ialpha, ibeta           current measured one control step before, pu of ibez
%   omega_hat               speed of the EKF block, pu of omega_bez
%   psi_alpha_hat, psi_beta_hat   flux of the EKF, pu of psi_m
%   u_scale                 udc_nom/ubez: voltage in pu of udc_nom -> voltage in pu of ubez
%   L_norm, R_norm          inductance and resistance seen by the bridge: q axis of the machine plus the output
%                           inductor, pu of the machine (the EKF uses the same values)
%   omega_bez               [rad/s] electrical base pulsation
%   k_kalman, k_sw          weights of the current error and of the number of legs that change state
%   ts_inv                  [s] control step
% Outputs:
%   ualpha, ubeta           voltage of the vector applied in the last step, pu of ubez. The Unit Delay after
%                           this block makes it the vector of two steps before: the vector that acted before the
%                           current measured one step before, the pair the EKF needs.
%   cmg_g1 ... cmg_g6       gate commands (0 or 15 V) of a+, a-, b+, b-, c+, c- (the dead time is added in
%                           the inverter by the gate generator)
%
% Timing: current and DC voltage are measured one control step before, and the vector chosen now acts until
% the next step. So the control first computes the current of now with the vector of the last step (delay
% compensation), then chooses the vector for the next step.

    persistent states_table u_last s_last

    if isempty(states_table)
        % the 8 states [leg a, leg b, leg c]: 1 = upper switch on, 0 = lower switch on
        states_table = [0 0 0; 1 0 0; 1 1 0; 0 1 0; 0 1 1; 0 0 1; 1 0 1; 1 1 1];
        u_last = [0; 0];            % voltage of the vector applied in the last step, pu of ubez
        s_last = [0 0 0];           % legs of the vector applied in the last step
    end

    Vdd = 15;
    inv_sqrt3 = 0.577350269189626;
    ts_pu = ts_inv*omega_bez;
    g = ts_pu/L_norm;

    % back-EMF of the EKF flux, e = d(psi)/dt = j*omega*psi (pu of ubez)
    ealpha = -omega_hat*psi_beta_hat;
    ebeta = omega_hat*psi_alpha_hat;

    % 1. Delay compensation: current of now, from the current measured one step before and the vector
    %    applied since then
    ialpha0 = ialpha + g*(u_last(1) - R_norm*ialpha - ealpha);
    ibeta0 = ibeta + g*(u_last(2) - R_norm*ibeta - ebeta);

    % 2. Prediction and cost of the 8 states (the first state with the lowest cost wins)
    minJ = 1e18;
    best_idx = 1;
    ua_best = 0.0;
    ub_best = 0.0;

    for n = 1:8
        sa = states_table(n,1);
        sb = states_table(n,2);
        sc = states_table(n,3);

        % alpha/beta voltage of the state, pu of ubez (leg voltages sa*vdc, sb*vdc, sc*vdc)
```



```
        ua = u_scale*vdc*(sa - 0.5*sb - 0.5*sc)*2/3;
        ub = u_scale*vdc*(sb - sc)*inv_sqrt3;

        % current at the next step
        ia_p = ialpha0 + g*(ua - R_norm*ialpha0 - ealpha);
        ib_p = ibeta0 + g*(ub - R_norm*ibeta0 - ebeta);

        % legs that change state from the vector of the last step
        nsw = abs(sa - s_last(1)) + abs(sb - s_last(2)) + abs(sc - s_last(3));

        % cost
        J = k_kalman*((ialpha_ref - ia_p)^2 + (ibeta_ref - ib_p)^2) + k_sw*nsw;

        if J < minJ
            minJ = J;
            best_idx = n;
            ua_best = ua;
            ub_best = ub;
        end
    end

    % 3. Voltage for the observers (vector of the last step), then the memory of the applied vector
    ualpha = u_last(1);
    ubeta = u_last(2);
    u_last = [ua_best; ub_best];
    s_last = states_table(best_idx, :);

    % 4. Gate commands of the chosen state
    ga = states_table(best_idx, 1);
    gb = states_table(best_idx, 2);
    gc = states_table(best_idx, 3);

    cmg_g1 = Vdd*ga;
    cmg_g2 = Vdd*(1 - ga);
    cmg_g3 = Vdd*gb;
    cmg_g4 = Vdd*(1 - gb);
    cmg_g5 = Vdd*gc;
    cmg_g6 = Vdd*(1 - gc);
end
```